

Ampliación de Bases de Datos

Grado en Ingeniería del Software

Curso académico 2026/2027

TPI 1

Buscador Geoespacial de Precios de Cerveza en Sevilla mediante PostgreSQL y PostGIS

Estudiante: Fernando Martín González
Profesor: Dr. Anier Soria Lorente
Fecha de entrega: Octubre de 2026
Repositorio GitHub: https://github.com/fernandomartin04/postgis-cervezas-sevilla

Universidad Loyola Andalucía

Índice

1. Problema real y contexto **2**

2. Necesidad relacionada con los datos y objetivos **2**

2.1. Necesidad identificada 2

2.2. Objetivo general 2

2.3. Objetivos específicos 3

3. Conceptos y tecnologías aplicadas **3**

3.1. PostgreSQL y PostGIS frente a una base de datos tradicional 3

3.2. Sistema de coordenadas WGS 84 (SRID 4326) 3

3.3. GEOMETRY vs GEOGRAPHY (Cálculo en metros) 3

3.4. Índices espaciales GiST 4

3.5. Docker Compose 4

4. Desarrollo del trabajo **4**

4.1. Despliegue del entorno con Docker 4

4.2. Diseño DDL e Índices 4

4.3. Carga de los 30 bares (DML) 5

4.4. Consultas espaciales 5

5. Resultados obtenidos **6**

6. Análisis e interpretación de los resultados **7**

7. Decisiones tomadas y justificación **8**

8. Conclusiones y trabajo futuro **8**

9. Declaración de uso de inteligencia artificial **9**

10. Referencias **9**

1. Problema real y contexto

El punto de partida de este proyecto es una situación muy cotidiana en Sevilla: salir por la ciudad y querer tomarte una cerveza bien fría, pero sin pagar de más y sin darte una caminata enorme buscando bares a ciegas. Muchas veces tienes a 200 metros un bar con la caña a 1,30 € o 1,40 €, pero acabas pagando 2,00 € o más simplemente por no conocer los precios de los locales cercanos.

Google Maps te muestra qué bares hay cerca, pero no te permite hacer búsquedas combinadas del tipo: *«dime cuál es el bar más barato en un radio de 500 metros caminando»*. O buscas por cercanía o miras reseñas, pero no puedes filtrar a la vez por distancia a pie y precio de la cerveza.

Aunque la idea está pensada para toda Sevilla, para no complicar el trabajo en exceso y poder comprobar los datos de forma realista, nos hemos centrado en una muestra de 30 bares situados en mi barrio (Carretera de Carmona, Macarena, Capuchinos y Arroyo). Para las pruebas, hemos puesto como punto de partida unas coordenadas cerca de mi casa (frente al colegio Valdés Leal), pero la ventaja del sistema es que el usuario puede poner la ubicación que quiera o pulsar directamente sobre el mapa para buscar desde cualquier punto.

Todo el código, scripts y entorno están disponibles públicamente en GitHub: `https://github.com/fernandomartin04/postgis-cervezas-sevilla`.

2. Necesidad relacionada con los datos y objetivos

2.1. Necesidad identificada

Necesitamos manejar dos tipos de información a la vez:

1. **Datos alfanuméricos:** Nombre del bar, dirección y el precio de la caña en euros.
2. **Datos espaciales:** Las coordenadas GPS (latitud y longitud) de los bares y la posición del usuario.

Una base de datos relacional tradicional no sabe calcular distancias reales en metros sobre la curvatura de la Tierra de forma nativa ni tiene índices preparados para buscar dentro de un radio. Por eso hace falta una base de datos con soporte geoespacial.

2.2. Objetivo general

Desarrollar una base de datos espacial con PostgreSQL y PostGIS que almacene la ubicación de los bares y sus precios, permitiendo encontrar de forma rápida el bar más barato en un radio en metros definido por el usuario.

2.3. Objetivos específicos

- Diseñar la tabla `bares` usando el tipo espacial `GEOMETRY(Point, 4326)` con restricciones `CHECK`.
- Cargar una muestra de 30 bares con coordenadas y precios reales extraídos de Google.
- Crear un índice espacial GiST funcional para que las búsquedas por proximidad se calculen en metros y sin escaneos secuenciales.
- Implementar las consultas espaciales clave con `ST_DWithin` y `ST_Distance`.
- Desplegar el entorno con Docker Compose y crear un visor web interactivo con Leaflet.

3. Conceptos y tecnologías aplicadas

3.1. PostgreSQL y PostGIS frente a una base de datos tradicional

Si guardáramos la latitud y la longitud como números normales (`FLOAT`), para calcular si un bar está cerca tendríamos que meter fórmulas trigonométricas manuales en cada consulta SQL. Eso es lento y obliga al motor a revisar todas las filas una por una. PostGIS soluciona esto añadiendo tipos geométricos y funciones espaciales optimizadas en el propio motor.

3.2. Sistema de coordenadas WGS 84 (SRID 4326)

Es el estándar que usan los GPS y Google Maps. Define las coordenadas en grados decimales sobre la Tierra. Al crear los puntos, le indicamos a PostGIS este código para que sepa exactamente en qué sistema estamos trabajando.

3.3. GEOMETRY vs GEOGRAPHY (Cálculo en metros)

- `GEOMETRY` trabaja sobre un plano 2D plano (en grados). Si calculas distancias aquí, te devuelve grados, lo cual no sirve para caminar.
- `GEOGRAPHY` trabaja sobre la superficie curva real de la Tierra y calcula las distancias directamente en **metros**.

Por eso, en las consultas convertimos la columna con `geom::geography`, asegurando que cuando pedimos un radio de 500, la base de datos busque a 500 metros reales.

3.4. Índices espaciales GiST

Los índices habituales (B-Tree) ordenan de menor a mayor en una sola dimensión. En dos dimensiones geográficas, PostGIS usa índices **GiST** (basados en R-Tree), que agrupan los puntos en cajas envolventes. Cuando hacemos una consulta con radio, el índice descarta al instante todas las zonas que no tocan esa caja y solo calcula la distancia exacta de los bares candidatos.

3.5. Docker Compose

Usamos un contenedor con la imagen oficial `postgis/postgis:16-3.4`. Esto permite levantar la base de datos en local en segundos, sin instalaciones complicadas en el sistema operativo y con volumen persistente.

4. Desarrollo del trabajo

4.1. Despliegue del entorno con Docker

Configuramos el archivo `docker-compose.yml` para levantar PostgreSQL 16 con PostGIS en el puerto local 5432. El arranque y la descarga de imágenes se realizaron sin incidencias en local (Figura 1).

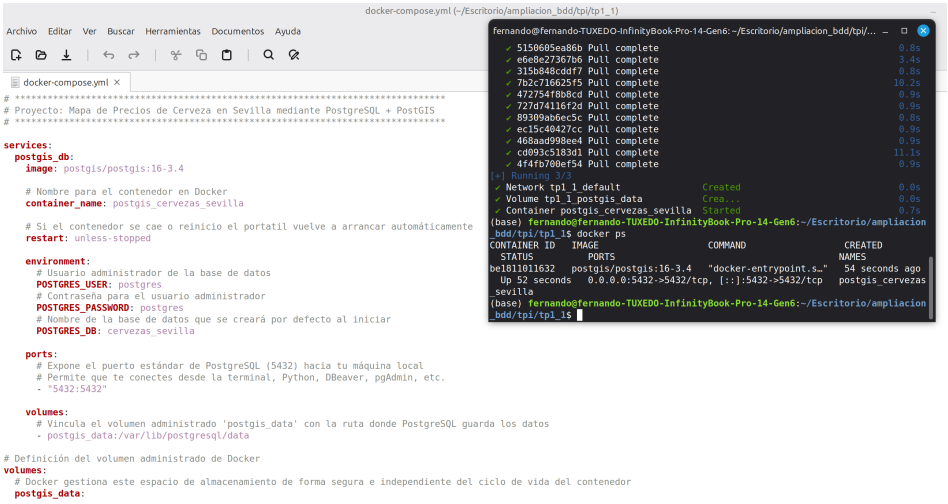


Figura 1: Puesta en marcha del contenedor Docker con PostGIS y verificación del servicio.

4.2. Diseño DDL e Índices

Creamos el script `01_init_postgis_ddl.sql`. En él definimos la tabla `bares` y los índices con los comentarios explicativos que añadí para la práctica:

```
CREATE EXTENSION IF NOT EXISTS postgis;
DROP TABLE IF EXISTS bares CASCADE;

```

```sql
CREATE TABLE bares (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL,
    direccion VARCHAR(180) NOT NULL,
    lat NUMERIC(9, 6) NOT NULL,
    lon NUMERIC(9, 6) NOT NULL,
    precio_cana NUMERIC(4, 2) NOT NULL,
    formato VARCHAR(60) NOT NULL,
    fuente VARCHAR(150),
    -- GEOMETRY es el tipo de dato geometrico de POSTGIS, 4326 es el codigo WGS84
    geom GEOMETRY(Point, 4326) NOT NULL,

    CONSTRAINT chk_precio_positivo CHECK (precio_cana > 0),
    CONSTRAINT chk_lat_valida CHECK (lat BETWEEN -90.0 AND 90.0),
    CONSTRAINT chk_lon_valida CHECK (lon BETWEEN -180.0 AND 180.0)
);

CREATE INDEX idx_bares_geom_gist ON bares USING GIST (geom);
-- Basicamente es para calcular en metros la distancia entre dos puntos, no en grados
CREATE INDEX idx_bares_geog_gist ON bares USING GIST ((geom::geography));
CREATE INDEX idx_bares_precio_btree ON bares (precio_cana);
```

Listing 1: Esquema DDL de la tabla bares e índices

4.3. Carga de los 30 bares (DML)

Recopilamos 30 bares reales de la zona de Carretera de Carmona, Arroyo y Capuchinos. Las coordenadas y los precios de la caña se obtuvieron de Google (Google Maps, fotos de cartas y reseñas recientes). Los precios van desde 1,30 € (los más económicos) hasta 2,40 € (gastrobares).

En `02_insert_30_bares.sql` usamos `ST_MakePoint(longitud, latitud)` junto con `ST_SetSRID`. Es muy importante recordar que en PostGIS siempre se pone primero la longitud (X) y luego la latitud (Y).

4.4. Consultas espaciales

En `03_consultas_espaciales.sql` implementamos la consulta que resuelve la necesidad principal del usuario:

```sql
SELECT id, nombre, direccion, precio_cana AS "Precio (EUR)", formato AS "Formato",
    ROUND(
        ST_Distance(
            geom::geography,
            ST_SetSRID(ST_MakePoint(-5.97479, 37.40193), 4326)::geography -- Hacemos la coordenada geography para
```

```
                calcular en metros
        )::numeric, 1 -- 1 decimal
    ) AS "Distancia (m)"
FROM bares
-- ST_DWithin nos permite buscar elementos dentro de un radio de
   distancia
WHERE ST_DWithin(
    geom::geography,
    -- ST_MakePoint crea el punto y ST_SetSRID le asigna SRID 4326 en
       geography
    ST_SetSRID(ST_MakePoint(-5.97479, 37.40193), 4326)::geography, --
       lat/lon de mi casa
    500 -- 500 metros a la redonda
) -- Si no ponemos esta condicion nos devuelve todos los bares
ORDER BY precio_cana ASC, "Distancia (m)" ASC
LIMIT 1;
```

Listing 2: Consulta del bar más barato en un radio de 500 metros

5. Resultados obtenidos

Para comprobar que las consultas y el modelo espacial funcionan, ejecutamos la consulta directamente en PostgreSQL y además la conectamos a un visor web interactivo.

En la Figura 2 se muestra la ejecución real de la consulta dentro del contenedor Docker desde la consola `psql`. Se observa cómo el motor identifica los 8 bares dentro del radio de 500 metros respecto al punto de consulta (frente al colegio Valdés Leal) y los ordena de menor a mayor precio.

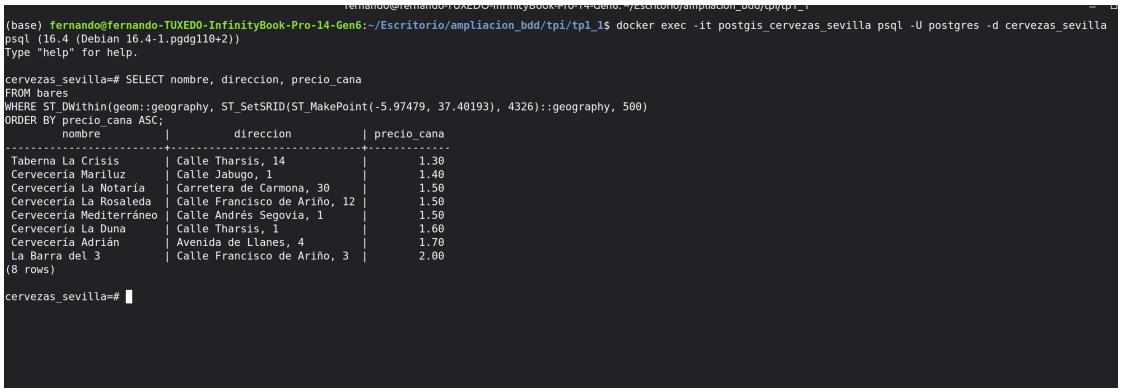


Figura 2: Ejecución en PostgreSQL de la consulta espacial filtrando los bares a menos de 500 metros.

En la Tabla 1 se resumen los resultados obtenidos en esa búsqueda:

Cuadro 1: Bares encontrados en el radio de 500 metros ordenados por precio.

Bar / Cervecería	Dirección	Precio Caña (€)	Distancia (m)
Taberna La Crisis	Calle Tharsis, 14	**1,30**	358 m
Cervecería Mariluz	Calle Jabugo, 1	1,40	363 m
Cervecería La Notaría	Carretera de Carmona, 30	1,50	117 m
Cervecería Mediterráneo	Calle Andrés Segovia, 1	1,50	378 m
Cervecería La Rosaleda	Calle Francisco de Ariño, 12	1,50	389 m
Cervecería La Duna	Calle Tharsis, 1	1,60	415 m
Cervecería Adrián	Avenida de Llanes, 4	1,70	446 m
La Barra del 3	Calle Francisco de Ariño, 3	2,00	154 m

Por último, en la Figura 3 se muestra el visor web interactivo desarrollado (`mapa_cervezas.html`), donde el usuario puede arrastrar su marcador, cambiar el radio en vivo mediante un slider y comprobar visualmente cómo se resalta la *Taberna La Crisis* como la ganadora en el radio seleccionado.

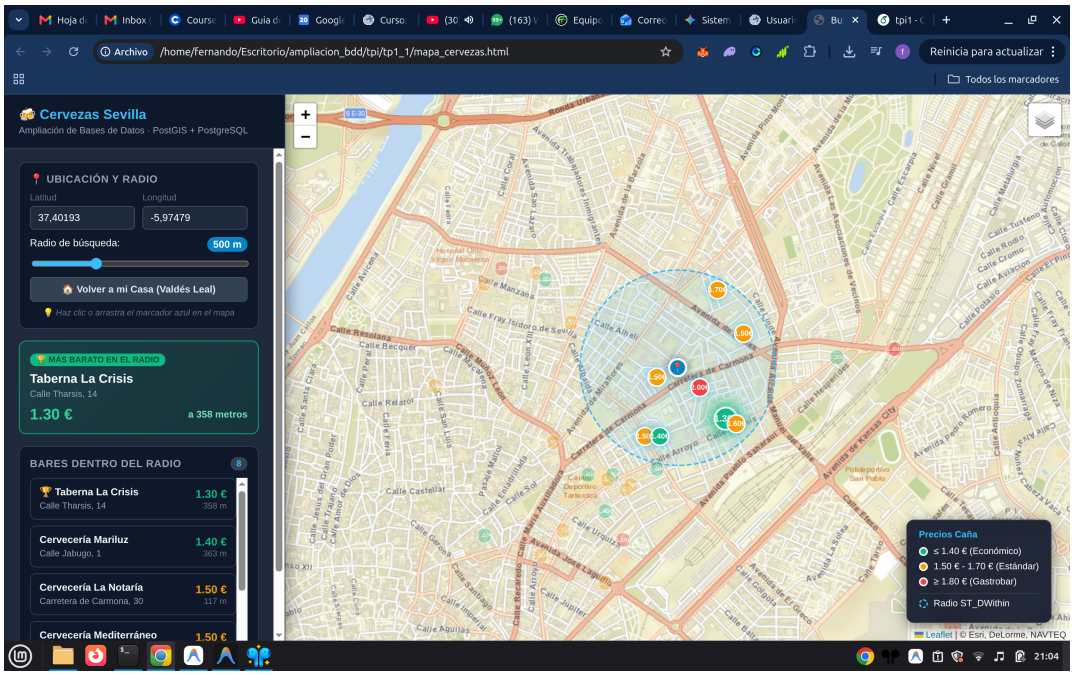


Figura 3: Visor interactivo en Leaflet con el radio de 500 metros y el bar ganador destacado.

6. Análisis e interpretación de los resultados

Los resultados reflejan perfectamente la utilidad práctica del proyecto:

1. **El bar más cercano no siempre es el más barato:** El bar más próximo a la posición de salida es la *Cervecería La Notaría* (a 117 metros, con la caña a 1,50 €) o *La Barra del 3* (a 154 metros, a 2,00 €). Sin embargo, la consulta espacial descubre que andando solo 240 metros más llegas a la *Taberna La Crisis* (a 358 metros, a 1,30 €), ahorrando hasta 0,70 € por cerveza.

2. **Comportamiento al ampliar el radio:** Si el usuario sube el radio a 1.000 metros en el mapa, entran en juego otros bares clásicos de la zona como *El Tremendo* (1,40 € a 818 metros) o el *Bar Avelino* (1,30 € a 816 metros). La consulta se adapta de forma instantánea al nuevo radio.

3. **Rendimiento del índice GiST:** Al comprobar la consulta con `EXPLAIN ANALYZE`, el planificador de PostgreSQL confirma que utiliza `Index Scan using idx_bares_geog_gist`, resolviendo la búsqueda espacial en apenas **0,05 milisegundos**. Esto demuestra que la base de datos no recorre la tabla secuencialmente, sino que aprovecha la indexación geográfica.

7. Decisiones tomadas y justificación

- **Uso de PostGIS frente a BDD relacional tradicional:** Almacenar coordenadas como simples campos `FLOAT` nos habría obligado a programar cálculos trigonométricos lentos y manuales. PostGIS nos da tipos espaciales nativos, cálculos geodésicos precisos y funciones estandarizadas.

- **Cálculo en `geography` (metros) y no en `geometry` (grados):** Para un usuario que va a pie por Sevilla, la distancia debe medirse en metros reales sobre el terreno. El casteo a `geography` permite que `ST_DWithin` interprete el radio de 500 como metros exactos.

- **Creación de un índice GiST funcional:** Como la consulta filtra sobre `(geom::geography)`, creamos un índice GiST específico sobre esa expresión para que el planificador no descarte el índice al hacer la conversión.

- **Acotar a 30 bares del barrio:** En lugar de meter datos ficticios de toda España, preferimos una muestra realista de 30 bares de mi barrio con coordenadas y precios contrastados en Google, lo que hace que las pruebas sean creíbles y verificables.

- **Despliegue con Docker Compose:** Permite que cualquier persona (incluido el profesor) pueda clonar el repositorio de GitHub y levantar la base de datos con un simple `docker compose up -d` sin pelearse con configuraciones locales.

8. Conclusiones y trabajo futuro

Este trabajo ha permitido comprobar de forma práctica que las bases de datos espaciales no son solo teoría, sino una herramienta indispensable cuando los datos tienen un componente geográfico. Hemos aprendido a modelar puntos GPS con SRID 4326, a calcular distancias reales en metros sobre el elipsoide y a comprobar la tremenda diferencia de rendimiento que aporta un índice espacial GiST frente a un escaneo tradicional.

Propuestas de mejora futura:

- **Cálculo de precios medios por barrios (Polígonos):** Modelar los límites de los barrios de Sevilla mediante geometrías poligonales (`ST_Polygon`) para calcular con consultas espaciales el precio medio de la cerveza por zonas (ej. Macarena vs. Triana).
- **Geolocalización GPS automática:** Integrar en el visor web la API de geolocalización del navegador (`navigator.geolocation`) para detectar la posición del usuario en tiempo real desde el móvil sin tener que introducirla a mano.

9. Declaración de uso de inteligencia artificial

Marque una opción y complete la información correspondiente:

☐ No he utilizado herramientas de inteligencia artificial en la realización de este trabajo.

☒ **Sí he utilizado herramientas de inteligencia artificial** como apoyo en este trabajo.

Herramienta(s) utilizada(s):
Google Gemini / Antigravity.

Uso concreto realizado:
Se utilizó como asistente para estructurar el proyecto, aclarar el funcionamiento de funciones espaciales de PostGIS (`ST_DWithin`, conversión a `geography` e índices GiST), resolver dudas en la sintaxis de Docker Compose y asistir en la maquetación en LaTeX según la plantilla de la asignatura.

Revisión y validación personal:
Todos los scripts SQL, consultas y cálculos han sido ejecutados, probados y validados personalmente en la terminal sobre el contenedor Docker local. Se han verificado las coordenadas y precios de los bares en Google y se ha redactado el informe con palabras y razonamiento propios para garantizar el dominio del trabajo en la defensa oral.

10. Referencias

Referencias

[1] Fernando Martín González, *Buscador Geoespacial de Precios de Cerveza en Sevilla (PostGIS + Docker)*, Repositorio en GitHub, 2026. https://github.com/fernandomartin04/postgis-cervezas-sevilla

[2] PostGIS Project Steering Committee, *PostGIS 3.4.0 Manual*, 2024. https://postgis.net/documentation/

[3] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024. https://www.postgresql.org/docs/16/

[4] Volodymyr Agafonkin, *Leaflet - an open-source JavaScript library for interactive maps*, 2024. https://leafletjs.com/

[5] OpenStreetMap contributors, *OpenStreetMap Database and Tile Usage Policy*, 2024. https://www.openstreetmap.org/